

Module 4 Project Rubric

TraceLens | Production-Ready Backend | AIE 500 - 10x Coding with AI

C2.4-C2.8; finalize C2.1/C2.2 | Completes C2.

Implement the first specified feature test-first, then prove test quality, safe refactoring, CI gates and container behavior.

Evidence to prepare

- Commit genuine failing tests before each first-feature increment; retain the red, green and refactor history.
- Reach meaningful pytest-cov coverage of at least 80%; test endpoint success/failures, storage, utilities and validation.
- Document a named refactor, before/after hashes and a green suite on both sides with unchanged public behavior.
- Create push/PR lint/test/coverage CI. Keep actual success, deliberate failure and recovery evidence, plus required merge-check proof.
- Build/run a backend Docker image and Compose development environment with hot reload; synchronize both specs and current documentation.

MUST PASS: C2.4 - Tests came first; C2.7 - Pipeline gates

A Not Yet returns this evidence group before its remaining criteria are reviewed.

Same competencies, independent project

Choose TraceLens **instead of PromptLab**. The videos demonstrate PromptLab; apply the methods independently. Both tracks require all 22 criteria to be Met, with no points, averaging or bonus credit.

No weekly capstone upload: keep one repository and authentic evidence for final review. Each module evidence group retains three attempts; returns do not consume one. See ASSESSMENT.md for revision, return and defense policy.

Version: TraceLens Criterion Set v1.1. Original project Modules 1-5 map to current Modules 2-6. The Syllabus governs policy; ALIGNMENT.md records clarifications.

Module 4 | Assessment criteria (1/3)

C2 - Specification-Driven Development & Automated Quality Engineering

C2.4-C2.8; finalize C2.1/C2.2 | Completes C2.

Criterion	Where your evidence lives	What earns "Met"	Common mistakes
C2.4 - Your tests came first MUST PASS	Commit history for your Module 4 feature	For each part of the feature, a commit containing a failing test precedes the commit containing its implementation. The Red-Green-Refactor rhythm is visible.	- All tests in one commit after the code. - Squashed history, so ordering can't be seen. - Tests committed first that assert nothing capable of failing.
C2.5 - Spec, tests and code line up	tests/, coverage report, specs/	Coverage at least 80% measured with pytest-cov across application code. Every implemented endpoint in your specs and API reference has meaningful passing success tests and documented failure-case tests where a failure is defined. Storage, utilities (including cost, pagination and stats) and model validation are covered. Planned second-feature endpoints are tested when implemented in Module 5.	- Endpoints built but not tested, or tested but not specified. - Coverage reached by tests that run code without asserting anything. - Failure cases specified but never tested.
C2.6 - Your refactor changed nothing visible	docs/refactor-note.md, before/after commits	A named code smell removed, the two commit hashes given, suite green on both sides, public interface and observable behaviour unchanged.	- Not naming the smell, or fixing something other than the one named. - Changing behaviour at the same time. - Editing tests to accommodate the refactor instead of using them to check it.

Source: TraceLens Criterion Set v1.1 (rubrics/CRITERIA.md), adapted from supplied v1.0 and aligned to the current PromptLab course package. All criteria are Met / Not Yet.

Module 4 | Assessment criteria (2/3)

C2 - Specification-Driven Development & Automated Quality Engineering

C2.4-C2.8; finalize C2.1/C2.2 | Completes C2.

Criterion	Where your evidence lives	What earns "Met"	Common mistakes
C2.7 - Your pipeline actually gates MUST PASS	.github/workflows/ci.yml, run logs, docs/ci-gate-evidence.md	Triggers on push and pull request, runs linting and tests with coverage, fails the build below 80%, passes on a clean clone, and demonstrably fails when a test is deliberately broken. Keep recovery evidence after restoring the test. Prove the required merge check separately in your repository settings.	- A workflow file with no successful run recorded. - A pipeline that runs tests but doesn't fail the build when they fail. - Passing locally but failing on a clean clone because config was never committed.
C2.8 - Your container builds and runs	backend/Dockerfile, docker-compose.yml, build output	The image builds from committed files with no manual steps. The container serves the API. docker-compose up --build brings up a working local environment, and the README explains Docker usage. Demonstrate local hot reload through the Compose development configuration.	- An image that builds but a container that exits or serves nothing. - A Dockerfile depending on files not in the repository. - A build needing undocumented manual steps.

Source: TraceLens Criterion Set v1.1 (rubrics/CRITERIA.md), adapted from supplied v1.0 and aligned to the current PromptLab course package. All criteria are Met / Not Yet.

Module 4 | Assessment criteria (3/3)

C2 - Specification-Driven Development & Automated Quality Engineering

C2.4-C2.8; finalize C2.1/C2.2 | Completes C2.

Criterion	Where your evidence lives	What earns "Met"	Common mistakes
C2.1 - Your specs are specific enough Final judgment in Module 4	specs/evaluations.md, specs/alert-rules.md	Each spec has overview and goals, user stories with acceptance criteria, data model changes, API endpoints with request and response shapes, and specified error conditions and edge cases. Every endpoint has at least one acceptance criterion you could write a test from directly.	- Requirements that name a feature without describing behaviour. - Specifying the happy path and leaving errors unstated. - Criteria you can't test, like "handles input correctly".
C2.2 - Your docs match your code Final judgment in Module 4	README.md, docstrings in models.py / api.py / storage.py / utils.py, docs/API_REFERENCE.md	Docstrings on every function and class across all four source modules, with parameters, returns and exceptions matching the implementation. API reference documents every endpoint that exists - including PATCH and your Module 4 feature - with request examples, sample responses and error formats. README setup steps work on a clean clone.	- API reference describing an earlier draft of the spec. - Docs not updated after Module 4's feature work. - Setup steps missing an environment variable or dependency. - Docstrings that restate the function name.

Final review. C2.1/C2.2 are repeated here for final judgment against the implemented first feature. Keep the second feature planned explicitly, then synchronize documentation when it is implemented in Module 5.

Source: TraceLens Criterion Set v1.1 (rubrics/CRITERIA.md), adapted from supplied v1.0 and aligned to the current PromptLab course package. All criteria are Met / Not Yet.